



Crypto Forecaster - BTC

Detailed Project Documentation & Technical Architecture

Developed by: Ahmed Fayad

April 18, 2026

Contents

1	Introduction	2
2	Key Features	2
3	Forecasting Models & Technical Specifications	2
3.1	1. Prophet	2
3.2	2. ARIMA (Auto-Regressive Integrated Moving Average)	3
3.3	3. Nixtla TimeGPT	4
3.4	4. ML Hybrid Model (ElasticNet + Random Forest)	5
4	Conclusion	6

1 Introduction

The **Crypto Forecaster** is a robust, interactive web application built with Streamlit. It is designed to forecast the future prices of Bitcoin (BTC) utilizing various advanced time-series analysis and machine learning models. The project bridges the gap between traditional statistical methods and modern foundational AI approaches to provide accurate, customizable forecasts.

2 Key Features

- **Interactive User Interface:** Built using Streamlit, allowing users to upload historical BTC data via CSV files seamlessly.
- **Dynamic Configuration:** Users can adjust the forecast horizon (from 7 up to 90 days) and select specific confidence intervals (80%, 90%, or 95%).
- **Technical Indicators:** Calculates and visualizes 30-Day Simple Moving Averages (SMA) and Exponential Moving Averages (EMA).
- **Rich Visualizations:** Employs Plotly to create interactive, dynamic charts showing historical data, projected trends, and confidence interval zones.
- **Detailed Tabular Outputs:** Provides users with a clear date-to-price forecasted dataframe, including lower and upper confidence bounds.

3 Forecasting Models & Technical Specifications

The core strength of the application lies in its diverse array of predictive models. Each model utilizes a fundamentally different mathematical approach to extract patterns from time-series data.

3.1 1. Prophet

Developed by Meta's Core Data Science team, Prophet is based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects.

Mathematical Representation: It frames forecasting as a curve-fitting exercise using the following additive model:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \tag{1}$$

- **$g(t)$ (Trend):** Models non-periodic changes. Prophet uses piecewise linear or logistic growth curve models for trend, intrinsically handling structural trend shifts (changepoints).
- **$s(t)$ (Seasonality):** Models periodic changes using Fourier series to approximate daily, weekly, and yearly fluctuations.
- **$h(t)$ (Holidays/Events):** Models the effects of specific external events.

- ϵ_t (**Error**): Represents idiosyncratic changes not accommodated by the model.

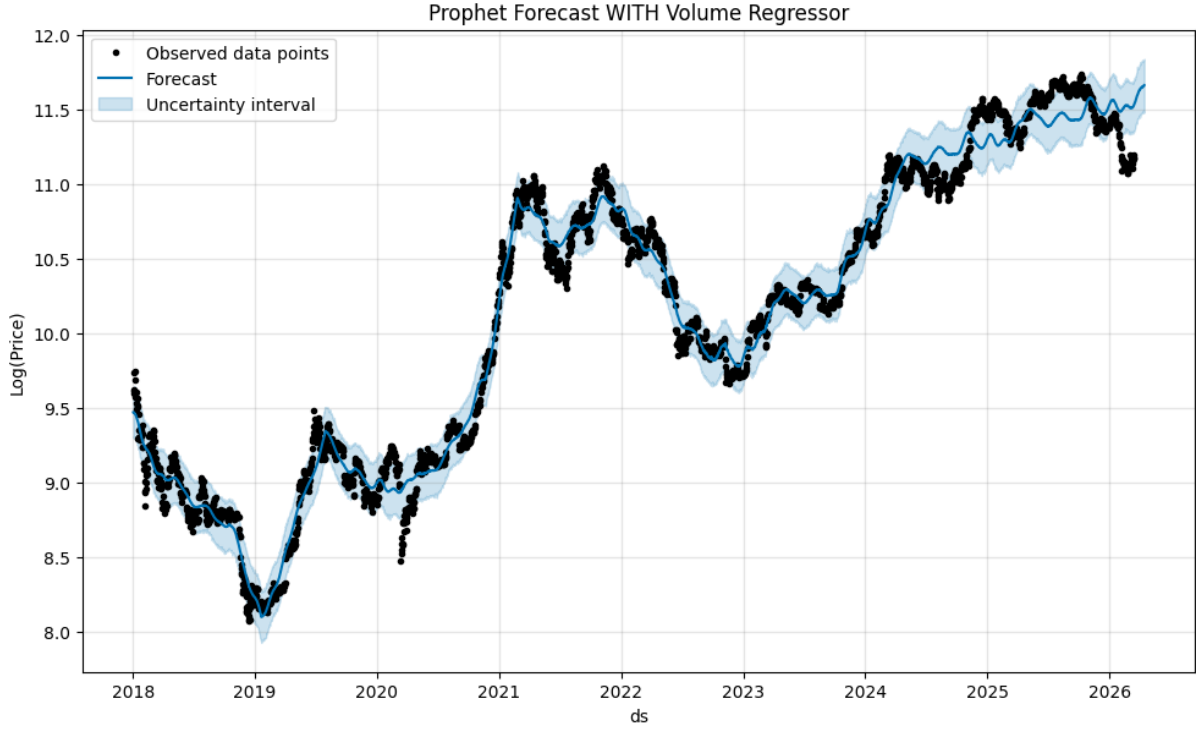


Figure 1: **Analysis of Prophet Forecast:** The black dots represent observed BTC prices. The solid blue line showcases the model's fitted trend and extrapolated forecast, while the shaded blue area indicates the uncertainty bounds. Prophet smoothly handles volatility and captures the overall directional momentum.

Limitations: While highly robust to missing data and seasonal shifts, Prophet treats forecasting purely as a curve-fitting problem. Its primary limitation is its inability to rapidly adapt to sudden, unprecedented market shocks (black swan events). Unless specifically configured with external regressors, it struggles to understand multivariate dependencies inside the volatile crypto market.

3.2 2. ARIMA (Auto-Regressive Integrated Moving Average)

ARIMA is a classical statistical method utilizing the `pmdarima` library's `auto_arima` tool, which automatically traverses the parameter space to find the optimal configuration (p, d, q) based on information criteria.

Technical Breakdown:

- **AR(p):** The current value depends linearly on its previous p values (lags).
- **I(d):** Applies differencing d times to stabilize the mean of the time series and achieve stationarity.
- **MA(q):** Models the current value as a linear combination of past forecast errors.

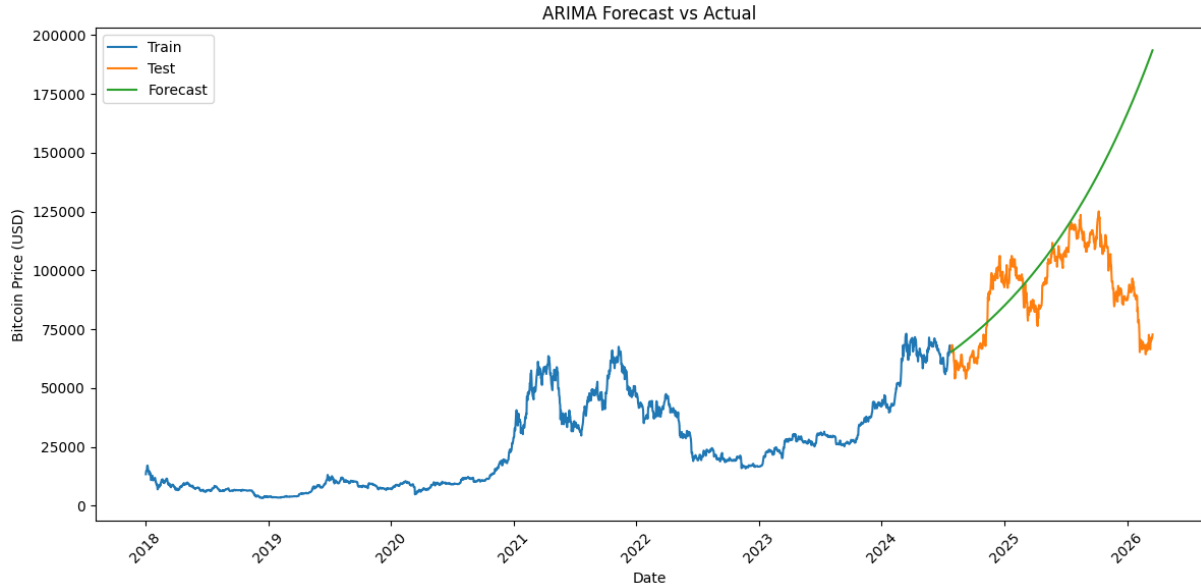


Figure 2: **Analysis of ARIMA Forecast vs Actual:** The graph highlights the behavior of traditional linear models on highly volatile crypto data. The blue line is training data, the orange represents unseen test data, and the green line is the ARIMA forecast. ARIMA captures a broad exponential upward trend but completely misses the sudden localized bursts and crashes (variance) found in the test dataset.

Limitations: As a purely linear model, ARIMA’s main limitation is its inability to capture complex, non-linear relationships. It typically reverts to the mean or follows a rigid exponential/linear trajectory over longer horizons (as seen in the flat/smooth green line), smoothing out the very volatility that traders seek to predict. Furthermore, it assumes constant variance (homoscedasticity), an assumption highly violated by Bitcoin.

3.3 3. Nixtla TimeGPT

TimeGPT marks a paradigm shift as the first foundation model (Generative Pre-trained Transformer) purposefully built for time-series forecasting.

Underlying Architecture:

- **Transformer Backbone:** Utilizes multi-head self-attention mechanisms to dynamically weigh the importance of different historical data points over irregular temporal distances.
- **Zero-Shot Inference:** TimeGPT utilizes transfer learning via its pre-training on billions of global time-series data points, allowing it to extrapolate future BTC prices without local fine-tuning.

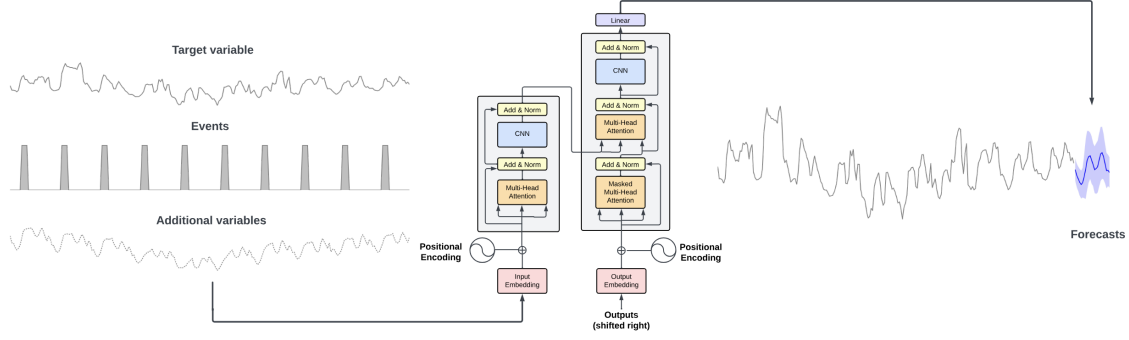


Figure 3: **Analysis of TimeGPT Architecture:** Sequential target variables are transformed by positional encodings and fed into an encoder-decoder architecture built upon stacked convolutional blocks and Multi-Head Attention mechanisms. This allows deep contextual awareness of long-term history before generating generative outputs.

Limitations: Because TimeGPT operates via a proprietary API, it introduces limitations surrounding data privacy, reliance on internet connectivity, and usage costs at scale. From an analytical perspective, it serves as a "black box" deep learning model—meaning it lacks the interpretability of standard statistical models, making it difficult for analysts to debug exactly *why* it predicted a specific market movement.

3.4 4. ML Hybrid Model (ElasticNet + Random Forest)

To solve the issue where tree-based models fail to extrapolate values beyond their training distribution, this application implements a custom detrended Hybrid Machine Learning pipeline.

Algorithmic Workflow:

1. **Feature Engineering:** Translating datetime into cyclical continuous features (sin/cos formatting), rolling means, and lag features ($t - 1, t - 3, t - 7$).
2. **Phase 1 (ElasticNetCV):** A linear regression model isolates the core directional trend while inherently dealing with feature multicollinearity via L1 and L2 penalties.
3. **Phase 2 (Random Forest):** A detrended residual series is created ($R = Y - \hat{Y}_{\text{trend}}$). A Random Forest Regressor is trained on these residuals to capture complex micro-movements.

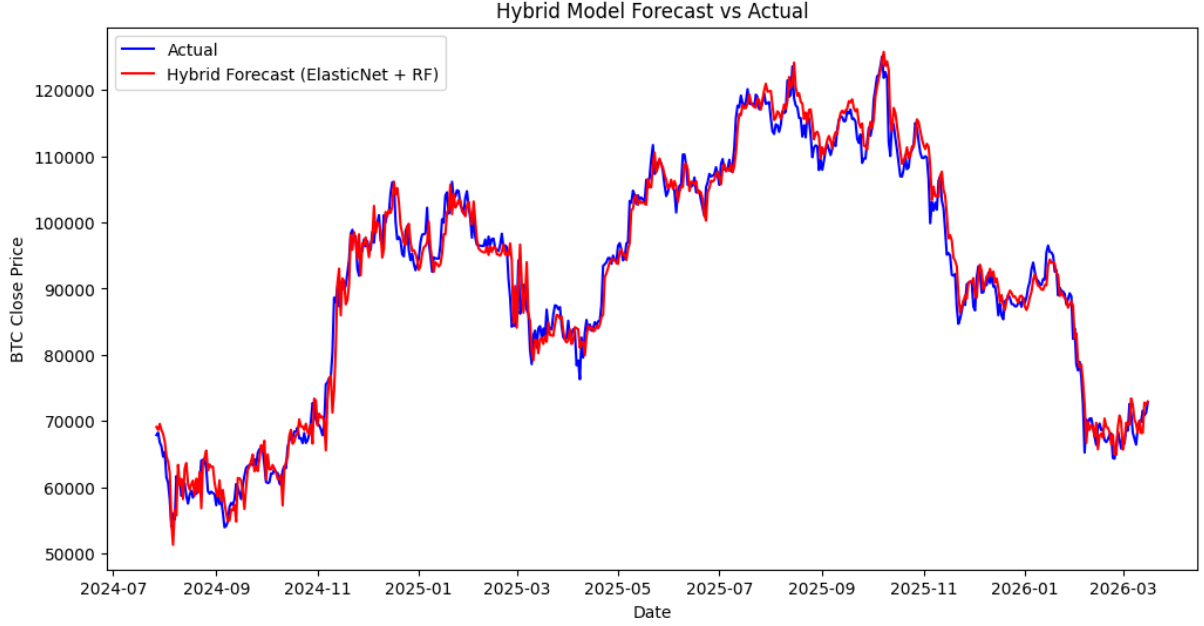


Figure 4: **Analysis of Hybrid Model Forecast vs Actual:** This graph illustrates the exceptional fit of the hybrid approach on the test dataset. The blue line represents the actual BTC closing prices, while the red line tracks the combined ElasticNet + Random Forest predictions. By mathematically removing the structural linear trend first, the Random Forest completely succeeds in tracking the high-frequency, non-linear market noise without degrading into a flat line.

Limitations: The primary limitation of this model is **Error Propagation** during forward multi-step forecasting. Because predicting step $t+2$ requires the predicted output of $t+1$ (iterative auto-regression), any small error made early in the forecast horizon cascades and magnifies the further out it predicts. Additionally, extreme shifts or structural breaks in the market can still trip the Random Forest if the residuals deviate wildly from the historical variance it learned during training.

4 Conclusion

By integrating statistical bases (ARIMA), additive curve-fitting (Prophet), deeply engineered sequential machine learning (ElasticNet + RF), and cutting-edge NLP-derived transformers (TimeGPT), the **Crypto Forecaster** covers all philosophical paradigms of time-series analysis allowing stakeholders to extract varying levels of market insights while balancing their respective limitations.